Software Engineering: Lab 3

Problem statement: Patient Health Record Consent Management System

Name: Jagriti Mohan

SRN: PES2UG24CS200

Section: 5th sem 'D'

1. Scenario Review

The system lets patients control who may view their electronic health records. Patients grant or revoke consent for specific providers and specific purposes (e.g. "Dr. Rao may view my cardiology records until 31 Dec 2026"). Every access to a health record must be checked against active consent, and every consent change and every access decision must be logged for regulatory audit.

Functional requirements:

- Patients can view, grant, modify, and revoke consent for named providers and specific record categories.
- Consent can be time-bound (auto-expiry) or purpose-bound.
- Clinical/EHR systems can request a patient's records and must be granted or denied access based on current consent.
- Patients and providers receive notifications when consent is granted, revoked, or about to expire.
- Every consent change and every access decision is recorded in an immutable audit log.

Non-functional requirements / challenges:

- Security & privacy: consent and health data are highly sensitive (HIPAA/GDPR-class data); authentication and authorization must be strictly enforced and isolated from other concerns.
- Compliance: audit logging must be tamper-resistant and always available, even if other parts of the system are degraded.
- Performance: consent checks sit on the critical path of clinical record access and must resolve quickly, even under load from multiple hospital systems.
- Reliability: a failure in a non-critical component (e.g. notifications) must never block a consent check or a clinical record request.
- Scalability: audit logging and notification volume can spike independently of consent-check volume (e.g. bulk provider onboarding).

2. Architectural Style Analysis

The three candidate styles were evaluated against the requirements above:

Architecture	Structure	Pros (general)	Cons (general)	Fit for this scenario
Layered	Horizontal tiers: Presentation, Business, Data	Simple to understand; clear separation of concerns	Tight coupling between layers; scaling is all-or-nothing; a bug in one layer can cascade	Weak fit — cannot isolate the consent engine, audit log, and notification channel independently, which this domain requires for compliance
Microservices	Small, independently deployable, independently scalable services	Independent scaling; fault isolation; each service can enforce its own security boundary; technology diversity	Higher operational complexity; network latency between services; eventual consistency challenges	Strong fit — consent, audit, notification and record-access concerns are naturally separable and have very different security, scaling and compliance needs
Client-Server	Centralized server, multiple thin clients	Simple deployment; centralized control; easy consistency	Single point of failure; scalability bottleneck; limited fault tolerance	Weak fit — a single server handling authentication, consent logic, audit, and notifications together is a single point of failure for a system that must never lose an audit trail or silently drop a consent revocation

3. Architecture Selection

Selected Architecture: Microservices

Reason 1 — Independent security boundaries: Authentication, consent decisions, audit logging, and notifications each carry different sensitivity and compliance obligations. Splitting them into separate services lets each be secured, deployed, and audited on its own boundary, instead of sharing a blast radius with unrelated code in a single layered application.

Reason 2 — Independent scaling under uneven load: Consent checks from clinical systems, audit-log writes, and patient notifications all spike at different times and rates (e.g. a hospital bulk-onboarding providers generates heavy audit and notification traffic but little consent-check traffic). Microservices let each concern scale independently, whereas a layered or client-server design would force the whole application tier to scale together.

Security Advantage: Because the Consent Management Service is the single authority that evaluates access, and it is isolated from the Health Record Access Service and the Authentication Service by well-defined APIs, a vulnerability or outage in one service (e.g. Notification Service) cannot be used to bypass consent checks or corrupt the audit trail — each service enforces its own access boundary rather than relying on a single shared codebase.

Performance Benefit: The Consent Management Service and Health Record Access Service can be scaled and cached independently of the Audit Logging and Notification services, keeping the consent-check path (which is latency-sensitive for clinicians) fast even when audit or notification volume spikes.

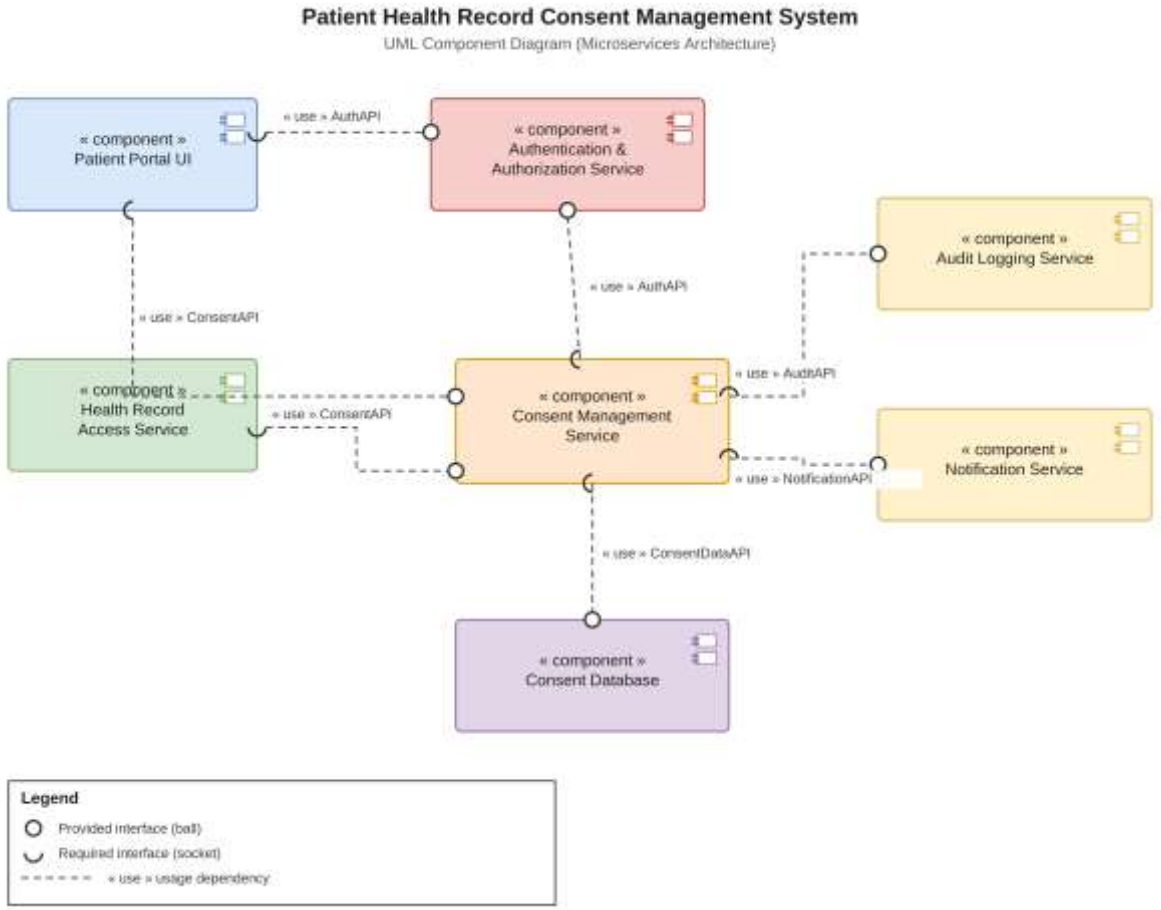
4. Component Identification

Seven components were identified (minimum of five required):

Component	Type	Responsibility	Provided Interface	Required Interface(s)
Patient Portal UI	Presentation	Lets patients view, grant, and revoke consent for named providers/purposes; shows record-access history	—	AuthAPI, ConsentAPI
Authentication & Authorization Service	Security	Verifies patient/provider identity, issues tokens, enforces role-based access control	AuthAPI	UserStore (internal)
Consent Management Service	Core business logic	Creates, updates, and revokes consent records; evaluates whether a given access request is currently authorised	ConsentAPI	AuthAPI, AuditAPI, NotificationAPI, ConsentDataAPI
Health Record Access Service	Integration	Gateway used by EHR/clinical systems to request patient records; must check consent before releasing any data	RecordAccessAPI	ConsentAPI
Audit Logging Service	Compliance	Immutably records every consent grant, revocation, and access decision for regulatory audit (e.g. HIPAA/GDPR)	AuditAPI	AuditDataStore (internal)
Notification Service	Communication	Notifies patients and providers when consent is granted, revoked, or about to expire	NotificationAPI	Email/SMS gateway (external)
Consent Database	Data	Persists consent records, scope, expiry, and history	ConsentDataAPI	—

5. Component Diagram

The full UML Component Diagram is also provided as an editable file: Patient_Consent_Component_Diagram.drawio (open at app.diagrams.net or in the draw.io desktop application.



Interfaces shown:

- Patient Portal UI → AuthAPI (Authentication & Authorization Service)
- Patient Portal UI → ConsentAPI (Consent Management Service)
- Consent Management Service → AuthAPI (Authentication & Authorization Service)
- Consent Management Service → AuditAPI (Audit Logging Service)
- Consent Management Service → NotificationAPI (Notification Service)
- Consent Management Service → ConsentDataAPI (Consent Database)
- Health Record Access Service → ConsentAPI (Consent Management Service) — the enforcement point: no record is released without a live consent check

6.Summary

(1) it gives independent, isolated security boundaries to authentication, consent decisions, audit logging, and notifications, which carry different sensitivity levels;

(2) it allows each concern to scale independently under the uneven, spiky load patterns typical of consent checks, audit writes, and notifications.

Security Advantage:

Isolating the Consent Management Service behind a well-defined API means every record request must pass through an explicit, auditable consent check, and a compromise or outage of a peripheral service (notifications, audit) cannot itself grant unauthorized access to health records.

Performance Benefit:

The latency-sensitive consent-check path (used directly by clinical systems) can be scaled and optimized independently of higher-volume but less time-critical audit and notification traffic, keeping clinician-facing record access fast.